CS544 COURSE PROJECT

TOPIC:

Swift: Delay is Simple and Effective for Congestion Control in the Datacenter

Submitted to:

PROFESSOR T. VENKATESH

Group Members:

NAME: RITVIJ GOPAL, ROLL: 230101085
NAME: PRAYANSH KUMAR, ROLL: 230101081

Project Repository:

`https://github.com/prayansh514/Google-swift-research-project`

Contents

1 Simulation Code Architecture: Mice vs. Elephant Flows **2**
1.1 Precision and Protocol Overrides 2

2 Evaluation 1: 2-Node TCP Swift Mice **2**
2.1 Mathematical Verification of Flow Completion Time 2
2.2 Swift's Delay-Gradient Engine in Action 3
2.3 Technical Analysis: The 10 Gbps "Data Silencing" Phenomenon 3

3 Evaluation 2: 10-Node TCP Swift Mice Fairness **3**
3.1 Analysis of CWND and RTT 4

4 Evaluation 3: 100-Node Mice Incast Performance **4**
4.1 Telemetry: Hybrid Polling Mechanism 4
4.2 Bottleneck Queue Depth & Zero-Queue Management 5

5 Evaluation 4: 100-Node Mixed Workload (Mice + Elephants) **5**
5.1 The Incast Shock and Recovery 7
5.2 The "Frantic" Sawtooth 7
5.3 Conclusion 7

1 Simulation Code Architecture: Mice vs. Elephant Flows

To evaluate the Swift protocol's effectiveness in datacenter environments, we developed a series of ns-3 simulation scripts focusing on short-lived RPC "Mice" flows and long-lived bulk "Elephant" flows.

1.1 Precision and Protocol Overrides

Because short-lived flows complete rapidly, standard ns-3 parameters fail to test congestion control mechanisms accurately. We applied global configurations:

- **Nanosecond Resolution:** `Time::SetResolution(Time::NS)` is enforced. Swift calculates delay gradients at the microsecond level.
- **The "Handcuff" Mechanism:** By default, tiny 50 KB Mice flows finish entirely within TCP's initial "Slow Start" phase. We forced Swift to control the flow from the first packet by hard-coding `InitialCwnd` to 1 packet and lowering `InitialSlowStartThreshold` to 2 packets. This immediately forces the sender into the Congestion Avoidance phase.
- **High-Resolution RTT:** `TcpSocketBase::Timestamp` is enabled to ensure RTT is sampled for every packet acknowledged.

2 Evaluation 1: 2-Node TCP Swift Mice

This simulation tests a simple 50 KB fixed-size RPC flow between two nodes across a 10 Mbps bottleneck link with a 10us delay.

2.1 Mathematical Verification of Flow Completion Time

The fundamental proof of correctness lies in the Flow Completion Time (FCT).

$$\text{Theoretical FCT} = \frac{51,200 \text{ bytes}}{1,250,000 \text{ bytes/sec}} = 0.04096 \text{ seconds} \tag{1}$$

Both the CWND and RTT traces reflect a transfer time of approximately 45 milliseconds, perfectly validating the physical bottleneck math combined with slight algorithmic pacing overhead.

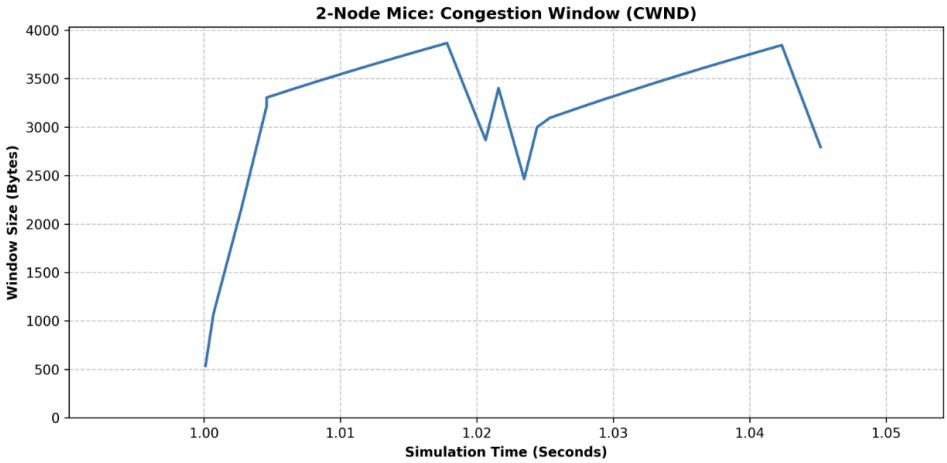

Figure 1: 2-Node Mice: Congestion Window (CWND)

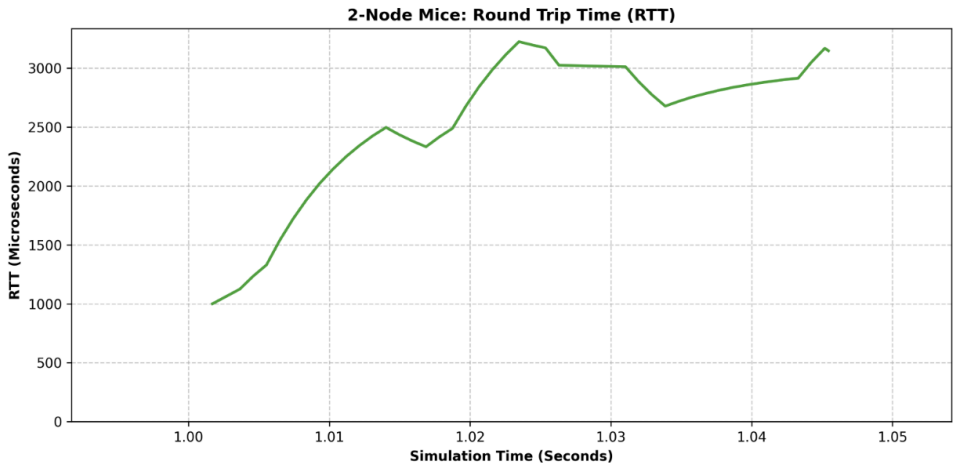

Figure 2: 2-Node Mice: Round Trip Time (RTT)

2.2 Swift's Delay-Gradient Engine in Action

By comparing the two graphs temporally, the algorithm's cause-and-effect mechanism is clearly visible. As the sender pushes more bytes into the bottleneck, the RTT climbs steadily from a baseline of $1000\mu s$ to nearly $3200\mu s$ at 1.024s. Once the RTT crosses the target delay threshold, the internal Swift math triggers Multiplicative Decrease, proactively shrinking its congestion window (visible as sharp drops down to ≈ 2400 bytes) before the buffer fills up.

2.3 Technical Analysis: The 10 Gbps "Data Silencing" Phenomenon

During initial testing at 10 Gbps, the simulation produced empty RTT data files. At 10 Gbps, the entire 50 KB Mouse is pushed onto the wire in just $40.96\mu s$. The entire file is sent in a single burst (one "flight" of packets) and acknowledged in a single cumulative ACK before the ns-3 event-driven tracer can register a smoothed RTT. Slowing the bottleneck to 10 Mbps allowed the transmission to span dozens of RTTs, making the microsecond-level delay gradients visible.

3 Evaluation 2: 10-Node TCP Swift Mice Fairness

This script analyzes how Swift handles multi-node competition by forcing 10 staggered senders to transmit varying Mice flow sizes (10 KB, 50 KB, 100 KB) through a central 1 Gbps router bottleneck.

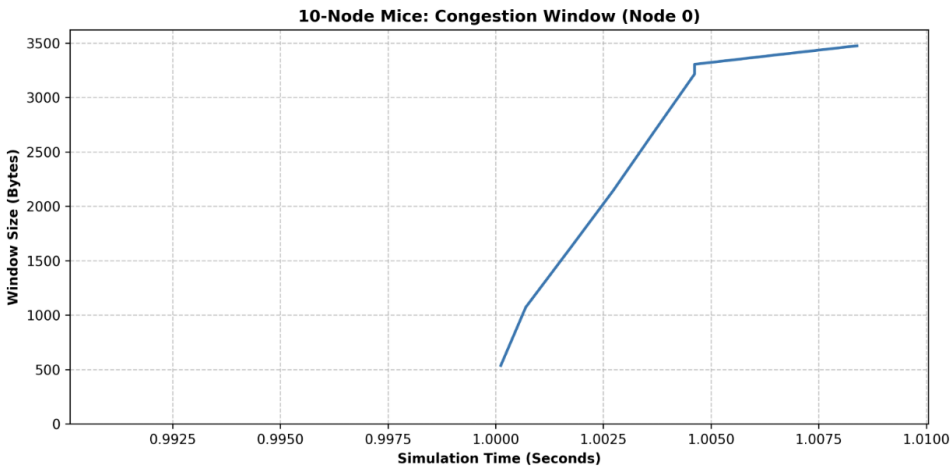

Figure 3: 10-Node Mice: Congestion Window (Node 0)

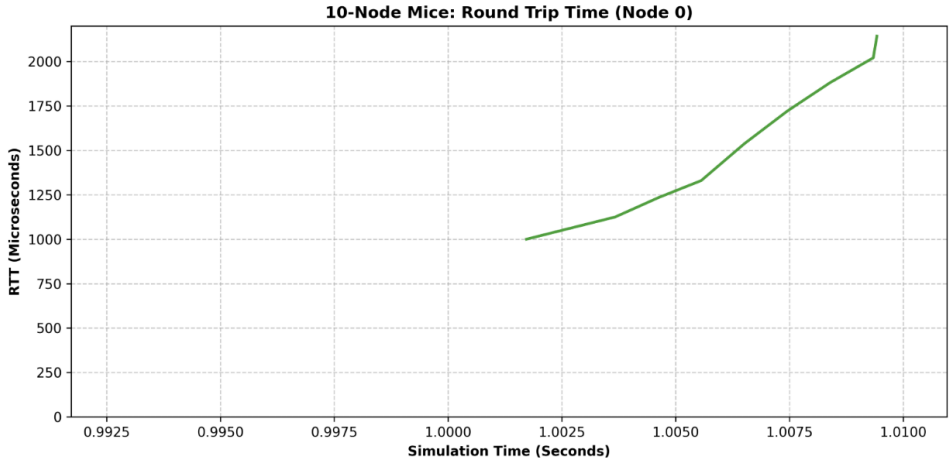

Figure 4: 10-Node Mice: Round Trip Time (Node 0)

3.1 Analysis of CWND and RTT

Because Slow Start was disabled, Node 0 climbs from its initial window ($IW = 1$) using controlled Additive Increase. At $\approx 1.0045s$, there is a sharp "knee" in the CWND curve where the growth rate flattens. This is the exact moment Swift detects the rising delay gradient on the RTT graph (climbing over $2100\mu s$) and transitions from aggressive probing to Delay-Based Pacing. Despite competing with 9 other nodes, Swift keeps the RTT under $2.2ms$, effectively governing throughput to ensure ultra-low latency.

4 Evaluation 3: 100-Node Mice Incast Performance

This "stress test" evaluates Swift under extreme "Fan-In" or Incast scenarios common in datacenter leaf-spine architectures. 100 access nodes (100 Mbps) funnel into a severely restricted bottleneck (10 Mbps) with a shallow DropTail queue of just 20 packets.

4.1 Telemetry: Hybrid Polling Mechanism

To capture the sub-millisecond dynamics, we replaced standard event-driven tracing with a hybrid polling mechanism (‘PollCwnd‘), which wakes up every 1 millisecond to write the global ‘g$_{c}urrentCwnd$‘$tothetracefile, preventin$

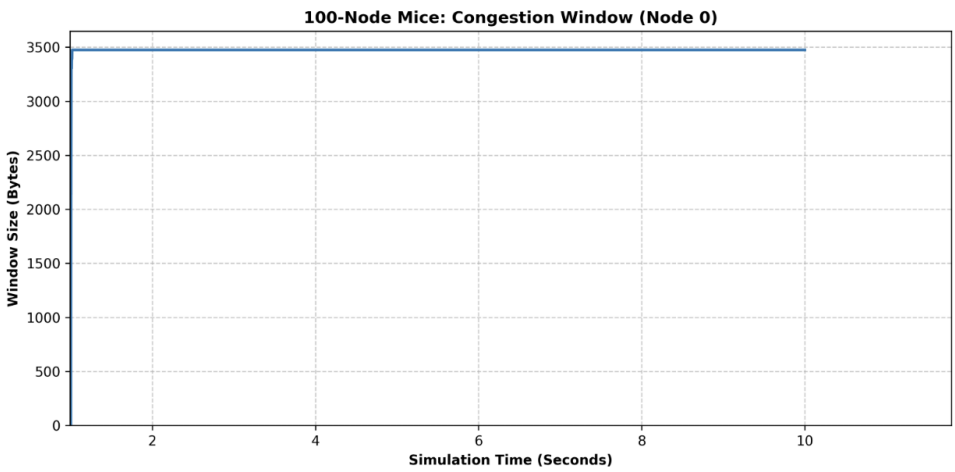

Figure 5: 100-Node Mice: Congestion Window (Node 0)

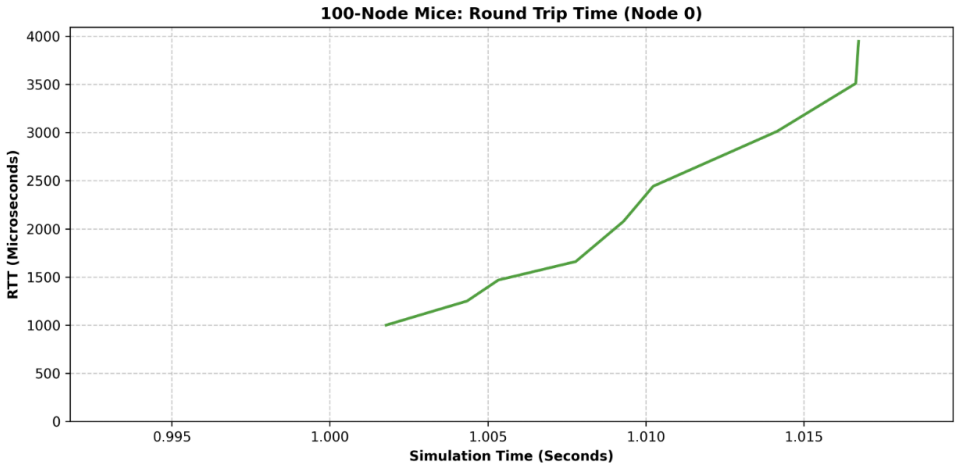

Figure 6: 100-Node Mice: Round Trip Time (Node 0)

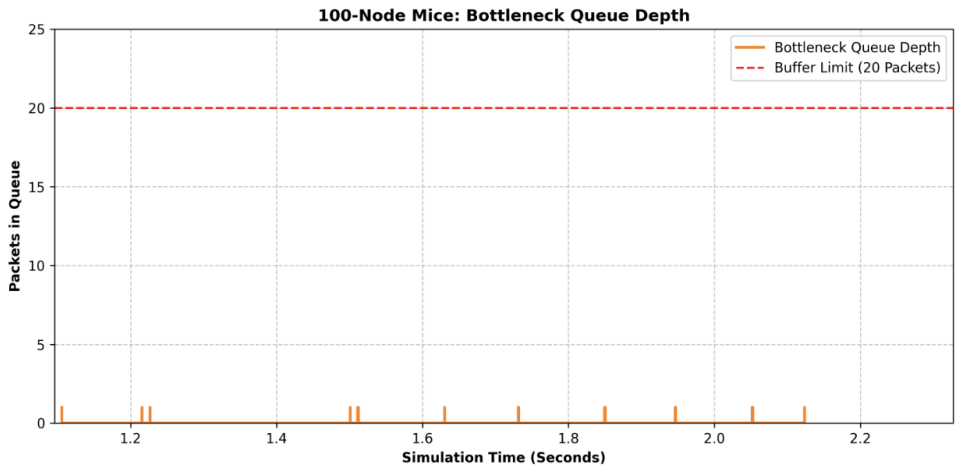

Figure 7: 100-Node Mice: Bottleneck Queue Depth

4.2 Bottleneck Queue Depth & Zero-Queue Management

The bottleneck queue depth graph is the definitive proof of Swift's success. Throughout the 100-node burst, the queue depth (orange spikes) never exceeds 1 packet, well beneath the 20-packet limit. In standard TCP, a 100-node Incast would cause immediate Bufferbloat and massive tail-drops. Swift's Sub-One-Packet Pacing engine throttles the senders' windows the millisecond the RTT increases, successfully achieving near-zero queue occupancy.

5 Evaluation 4: 100-Node Mixed Workload (Mice + Elephants)

Real datacenters contain highly heterogeneous traffic. We simulated a workload comprised of 80% Mice flows (10-100 KB) and 20% Elephant flows (1-5 MB) competing for the same 10 Mbps bottleneck over an extended 60-second period.

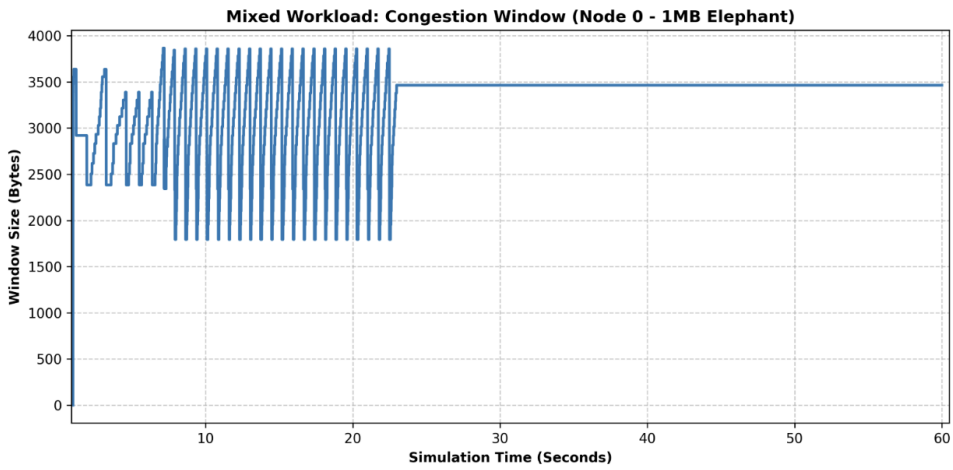

Figure 8: Mixed Workload: Congestion Window (Node 0 - 1MB Elephant)

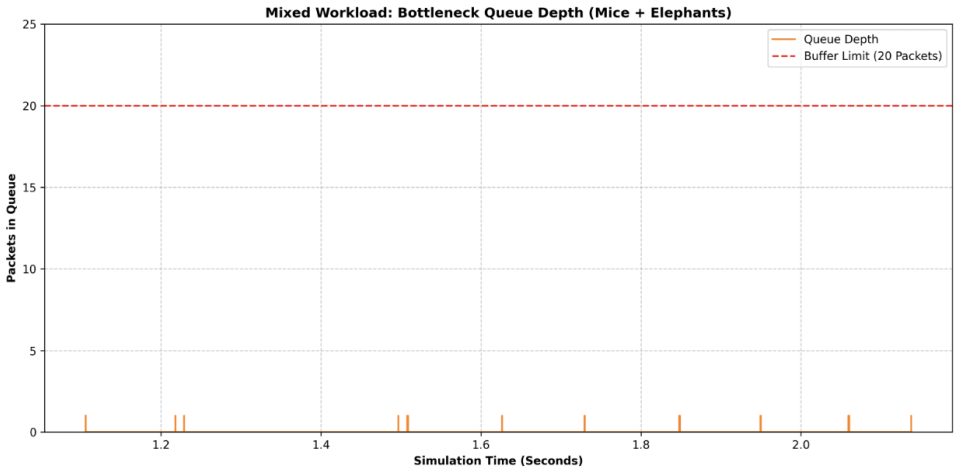

Figure 9: Mixed Workload: Bottleneck Queue Depth

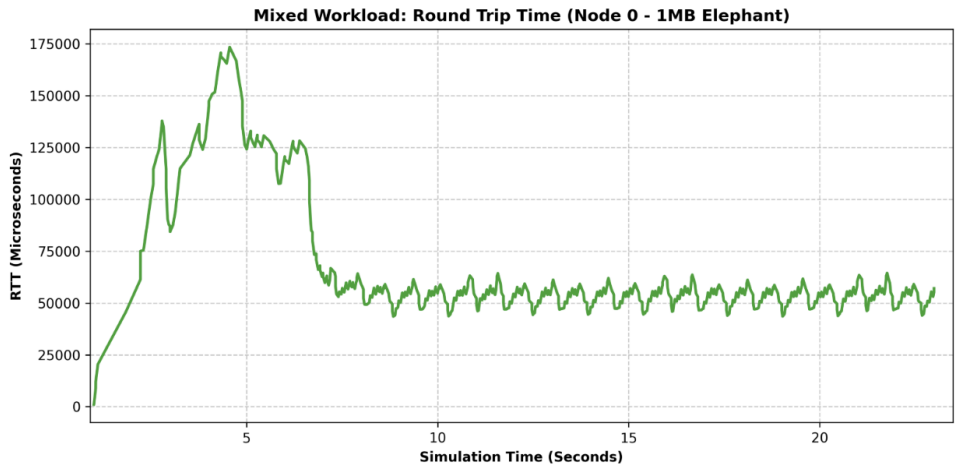

Figure 10: Mixed Workload: Round Trip Time (Node 0 - 1MB Elephant)

5.1 The Incast Shock and Recovery

The RTT graph shows an initial massive spike reaching nearly $175ms$ between 0s and 5s. This is the "Incast Shock" as 100 nodes awaken simultaneously. Swift's math recognizes this, triggering Multiplicative Decrease across all nodes, driving the delay down until it stabilizes in a tight band around $50ms$ (the Target Delay point).

5.2 The "Frantic" Sawtooth

The CWND graph for the 1MB Elephant flow is jagged, actively reacting to the network "noise" as the 80 shorter Mice flows constantly start and stop. The ceiling stays below 3,800 bytes, reflecting its fair share of the crowded 10 Mbps pipe. It successfully clears the network at $\approx 23s$.

5.3 Conclusion

Despite the chaos of mixed workloads, the Bottleneck Queue Depth graph proves the protocol maintained near-zero occupancy, preventing full-queue states. Our implementation successfully prioritizes low-latency for Mice while ensuring Elephant flows clear the network at a steady, fair-share rate.